UNIVERSITI MALAYSIA TERENGGANU

FACULTY OF COMPUTER SCIENCE AND MATHEMATICS

CSE3433
Software Architecture

Cloud-Native Learning Management System (LMS)
(Cloud-Native / Layered Architecture

Prepared by:

Group No.5

No.	Name	Matric No.
1	NUR MAS QASRINA BINTI MAS JAYA	S74706
2	AIDA HUSNA BINTI MUHAMAD ZAIDI	S76103
3	NURSYAHEERA BINTI MOHD HISHAM	S76276

Prepared for:
Dr Mohd Nor Hassan

COMPUTER SCIENCE (SOFTWARE ENGINEERING) WITH HONOUR
SEMESTER II 2025/2026

Table Of Content

1. PROBLEM DESCRIPTION

Current Cloud-Native LMS platforms still face several issues that affect both system performance and user experience. One of the main problems is the monolithic architecture, where all components are tightly connected, making it difficult to update or modify one part without affecting the whole system. These systems also have limited scalability, which leads to slow performance during peak usage such as exams or assignment submissions, and they may experience downtime due to limited infrastructure. In addition, poor system integration and weak API support make it hard to connect with external tools, reducing overall flexibility.

These issues impact all main users of the system, which are students, lecturers, and administrators. Students may struggle to access learning materials, miss deadlines due to system issues, and have a poor learning experience without real-time updates. Lecturers may face difficulties in managing courses, grading, and tracking student performance efficiently, which increases their workload. Administrators also face challenges in maintaining and scaling the system, higher operational costs, and difficulties in adding new features or integrating external systems.

2. SYSTEM OBJECTIVES

a. To develop a scalable and modular cloud-native Learning Management System (LMS).

   The LMS shall be designed in a cloud native manner with a system architecture that allows modular component development using APIs.

b. To ensure the efficiency of the interaction between system components through API-based communication.

   The interaction between the front-end, the backend service, and other external modules of the system shall be possible through communication using APIs.

c. To provide the availability and reliability of the learning platform.

   The system shall be designed to minimize downtime and ensure continuous access to learning resources through distributed deployment and cloud infrastructure capabilities.

d. To provide cross-platform capability for the learning platform.

   The system shall allow users (students and lecturers) to access the platform via web browsers across multiple devices without requiring platform-specific installation.

e. To enable easy maintainability and scalability of the system architecture.

   The system shall be structured into loosely coupled modules to allow easier maintenance, debugging, and future enhancement without affecting the entire system.

3. FUNCTIONAL REQUIREMENT

   a. User Management (Login/Register)

      i. User Registration: The system shall allow users to register with unique credentials and securely authenticate using industry-standard protocols.

      ii. Role-Based Access: The system shall assign student, instructor, or admin roles, ensuring users only access features relevant to their permissions.

   b. Course Management

      i. Course Lifecycle: Instructor shall have the ability to create, update, and archive courses, including managing their descriptions and associated metadata.

      ii. Curriculum Organization: The system shall enable instructors to upload and structure content formats such as PDF, video, and text into organized modules.

   c. Assignment & Submission

      i. Assignment Management: Instructors shall have the ability to define assignment parameters such as instructions, points, and deadlines.

      ii. Digital Submission: The system shall provide a secure interface for students to upload assignment files. Each submission must be timestamped to verify against the deadline.

   d. Notification System

i. Event Notification: The system shall automatically notify users of key activities, including new materials or posted grades.
ii. Deadline Reminders: Students shall receive automated reminders 24-48 hours before assignment deadlines.

e. Progress Tracking
   i. Completion Monitoring: The system shall track and record the status of individual lessons, assignments, and update students' overall completion percentage in real time.
   ii. Visual Indicators: Students shall see progress bars and status icons on their dashboard to reflect completed and pending tasks.

4. SELECTED ARCHITECTURE STYLE

The selected architecture style for this project is Cloud-Native Architecture combined with Layered Architecture supported by microservices concepts. This architecture is chosen because it best suits the requirements of a modern Learning Management System (LMS), where many users need continuous access to learning resources, assignment submissions, and real-time progress tracking.

a. Cloud-Native Architecture

   This architecture provides high scalability and reliability. The traditional monolithic LMS platforms frequently experience performance problems during times of high usage such test weeks or assignment deadlines. The suggested solution may maintain system availability with little downtime and expand resources dynamically based on user demand by utilizing cloud-native technologies. (*Understanding Cloud-native Apps*, n.d.)

b. Layered Architecture

This architecture is selected to divide the system into distinct and controllable levels including the Data Layer, Application Layer, and Presentation Layer. This separation of responsibilities improves maintainability, simplifies debugging, and allows future enhancements without affecting the entire system.

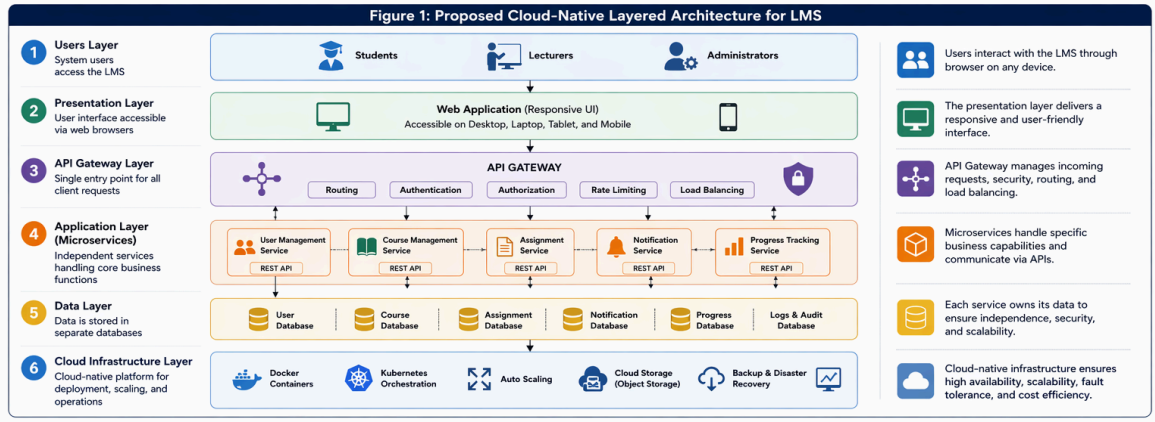

Figure 1: Proposed Cloud-Natived Architecture and Layered Architecture for LMS.

c. Microservices Architecture

Microservices architecture are applied within the application layer to divide the LMS into independent modules such as User Management, Course Management, Assignment Handling, Notification, and Progress Tracking services. Each service can be developed, deployed, and updated separately, improving flexibility and reducing system complexity.

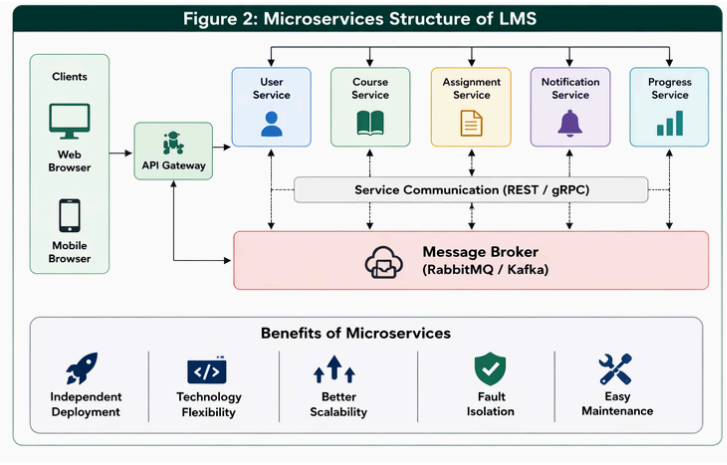

Figure 2: Microservices Structure of LMS

This selected architecture style also supports API-based communication between modules that makes it easier to integrate external tools or additional services in the future. As a result, the proposed architecture is more efficient, scalable, and future-ready compared to traditional monolithic LMS systems.

5. TEAM MEMBERS ROLES AND RESPONSIBILITIES

a. Team Structure and Roles Distribution

Table 1 : Team Members Roles & Responsibilities

Table 1 presents the roles assigned to each team member along with their respective responsibilities throughout the project development.

No.	Name	Role	Responsibilities
1.	Nur Mas Qasrina Binti Mas Jaya	Project Manager	Plan and manage overall project timeline and deliverables Coordinate team communication and task distribution Prepare architecture diagrams and documentation
2.	Aida Husna Binti Muhamad Zaidi	Backend Developer	Develop server-side logic and RESTful APIs using Node.js/ Spring Boot Manage database design and integration (MongoDB / SQLite) Handle version control using GitHub
3.	Nursyaheera Binti Mohd Hisham	Fronted Developer	Develop a responsive user interface using HTML, Bootstrap, or React Integrate frontend with backend APIs Ensure usability and user experience (UI/UX) quality

6. SYSTEM OVERVIEW

The Cloud-Native Learning Management System (LMS) is a distributed software platform created for seamless use by students, lecturers and admins. The platform ensures efficient arrangement of services such as access course materials, manage assignments, track learning progress, and receive real-time notification in an efficient and scalable. There are roles provided in the LMS. For example, students who are accessing learning material and submitting assignments. Other than that, lecturers who are creating and managing their courses, creating assignments and making them and admins who are managing system users.

The system provide functionalities in the platform such as User Management, Notification Service, Course Management, Assignment Submissions and Progress Tracking. These functionalities are implemented to ensure that teaching and learning processes become more efficient. This system also implements a Request-Response paradigm using the API Gateway as the interface for interaction between Web Fronted and the system. Data is stored in specialized databases where user information, course content and tracking information is stored. The cloud-native approach has been applied alongside the Layered architecture and microservices. The system consists of Presentation, Application and Data layers, which every layer carrying out specific responsibilities.

The microservices design pattern can be retained in the Application layer by which the basic functionality is broken down into various services such as User service, Course service, Assignment service, Notification service, and Progress Tracking service.

This involves communication between components through the RESTful API that ensures flexibility and ease of integration. The deployment occurs in a cloud environment and auto-scaling with high availability is guaranteed.

7. FUNCTIONAL REQUIREMENT
    a. User Management
        - FR - 1.1 : The system shall allow users to register an account by providing email, full name and password.
        - FR - 1.2 : The system shall provide a secure login interface that authenticates users with encrypted credentials.
        - FR - 1.3 : The system shall enforce Role Based Access Control, which ensures only authorized users can access specific pages such as the user list for admin.
    b. Course Management
        - FR - 2.1 : The system shall allow instructors to create new courses by entering a category, title and description.
        - FR - 2.2 : The System shall support uploading course materials in multiple formats, including PDF documents and MP4 video files.
        - FR - 2.3 : The system shall provide a search function that enables students to find courses using keywords or instructor names.
    c. Assignment & Submission
        - FR - 3.1 : The system shall allow instructors to set specific due dates and time limits for each assignment.
        - FR - 3.2 : The system shall permit students to upload assignment files with a maximum size limit.
        - FR - 3.3 : The system shall automatically block or reject a submission made after the deadline.
    d. Notification
        - FR - 4.1 : The system shall send automated notifications via email or dashboard when instructors post new course material.
        - FR - 4.2 : The system shall send a push notification to student 24 hours before an assignment deadline.
    e. Progress Tracking
        - FR - 5.1 : The system shall update student a percentage complete each time they mark a lesson as finished.
        - FR - 5.2 : The system shall display a visual progress bar on the student dashboard for every enrolled course.
        -
8. NON - FUNCTIONAL REQUIREMENT
    a. Availability
        - NFR - 1.1 : The system shall maintain an uptime 99.9% ensuring student can reliably access course material and submit assignments at any time.
        - NFR - 1.2 : The system shall implement an automated failover mechanism to guarantee service continuity in the event of a cloud server failure.

b. Scalability
    - NFR - 2.1 : The system shall support horizontal scaling which enabling it to handle sudden increase in concurrent user without performance degradation.
    - NFR - 2.2 : The database shall be capable of storing and retrieving data for up to 10,000 active users while maintaining acceptable latency levels.

c. Performance
    - NFR - 3.1 : The system shall deliver a response time of less than 2 seconds for standard page loads under normal traffic conditions.
    - NFR - 3.2 : File upload and download processes shall be optimized like a 20MB assignment file is completed within 10 seconds.

d. Security
    - NFR - 4.1 : all sensitive data, include user password and personal information shall be encrypted or equivalent industry standard before storage in the database.
    - NFR - 4.2 : The system shall enforce HTTP for all data transmission to safeguard against unauthorized interception of student and instructor information.

e. Maintainability & usability
    - NFR - 5.1 : The system architecture shall be modular which allows for seamless updates or the addition of new features without disrupting existing services.
    - NFR - 5.2 : The user interface shall be fully responsive by ensuring consistent functionality across smartphone, desktops and tablet,

9. ARCHITECTURE STYLE EXPLANATION

The proposed Learning Management System (LMS) adopts a combination of Cloud-Native Architecture, Layered Architecture, and Microservices Architecture to ensure high scalability, flexibility, and maintainability.

1. Cloud-Native Architecture

   This architecture makes it possible to use contemporary technologies like distributed services and containerization to deploy and manage the system in a cloud environment. Cloud-native concepts in this LMS enable the system to:

   - Adapt dynamically to user demand, particularly during busy times like exams and assignment turn-ins.
   - Ensure high availability through distributed deployment across multiple servers.

- Use load balancing and automated failover to reduce downtime.

The LMS is more dependable than conventional monolithic systems since it uses cloud infrastructure to support thousands of concurrent users without experiencing performance degradation.

2. Layered Architecture

   For this architecture, the three primary layers of the system's layered structure which divides duties into discrete levels are as follows:

   - Presentation Layer (Fronted)

     This layer handles user interaction through web interfaces. It is developed using technologies such as HTML, Bootstrap, or React. Users such as students, lecturers, and administrators interact with the system through this layer.
   - Application Layer (Backend Services)

     This layer processes business logic and handles communication between the frontend and the database. It includes API services responsible for user authentication, course management, assignment processing, and notifications.
   - Data Layer (Database)

     This layer stores and manages all system data, including user information, course content, assignments, and progress records. It uses databases like MongoDB or SQLite.

   This separation enhances system maintainability and makes debugging easier. It also allows independent upgrades without impacting the entire system.

3. Microservices Architecture

   The system is further subdivided into separate microservices within the application layer. Every microservice is responsible for a specific function of the LMS.

   - **User Management Service**: Handles registration, login, and role-based access control.

- **Course Management Service**: Manages course creation, updates, and content organization.
- **Assignment Service**: Handles assignment creation, submission, and deadline validation.
- **Notification Service**: Sends alerts and reminders to users.
- **Progress Tracking Service**: Monitors and updates student learning progress.

Each microservice can be developed, deployed, and scaled independently. This lowers the complexity of the system and enables quicker upgrades without interfering with other services.

4. API-Based Communication

   The API Gateway serves as a central entry point, handling requests, authentication and routing to the proper services. The use of RESTful APIs guarantees loose coupling between frontend and backend components. It makes the intergartion more easier with external systems such as third-party tools and standardized communication between microservices. (*API Management - Amazon API Gateway - AWS*, n.d.)

10. SERVICE / COMPONENT IDENTIFICATION

The LMS system is decomposed into several independent microservices within the application layer. Each service is responsible for a specific function to ensure modularity and scalability.

Table 2: Component / Service Identification

Table 2 illustrates the key microservices and their respective functionalities in the Cloud-Native LMS.

No.	Services	Description
1	User Management Service	Handles user registration, authentication, and role-based access control. It ensures secure login

		and manages user profiles.
2	Course Management Service	Responsible for creating, updating, and organizing course content. It allows instructors to upload materials and manage course structures.
3	Assignment Service	Manages assignment creation, submission, and validation. It ensures that submissions are timestamped and comply with deadlines.
4	Notification Service	Sends real-time alerts and reminders to users via email or dashboard notifications. It is triggered by events such as new materials or upcoming deadlines.
5	Progress Tracking Service	Tracks user activity and calculates course completion percentages. It updates progress indicators displayed on the user dashboard.
6	API Gateway	Acts as a single entry point for all client requests and routes them to the appropriate microservices. It also handles authentication and request validation.
7	Database Service	Stores all system data including user information, course content, assignments, and progress records. It supports scalable and efficient data access.

These components are designed to work independently while communicating through APIs, supporting a scalable and maintainable system architecture.

11. ARCHITECTURE DIAGRAM

11.1 SYSTEM CONTEXT DIAGRAM

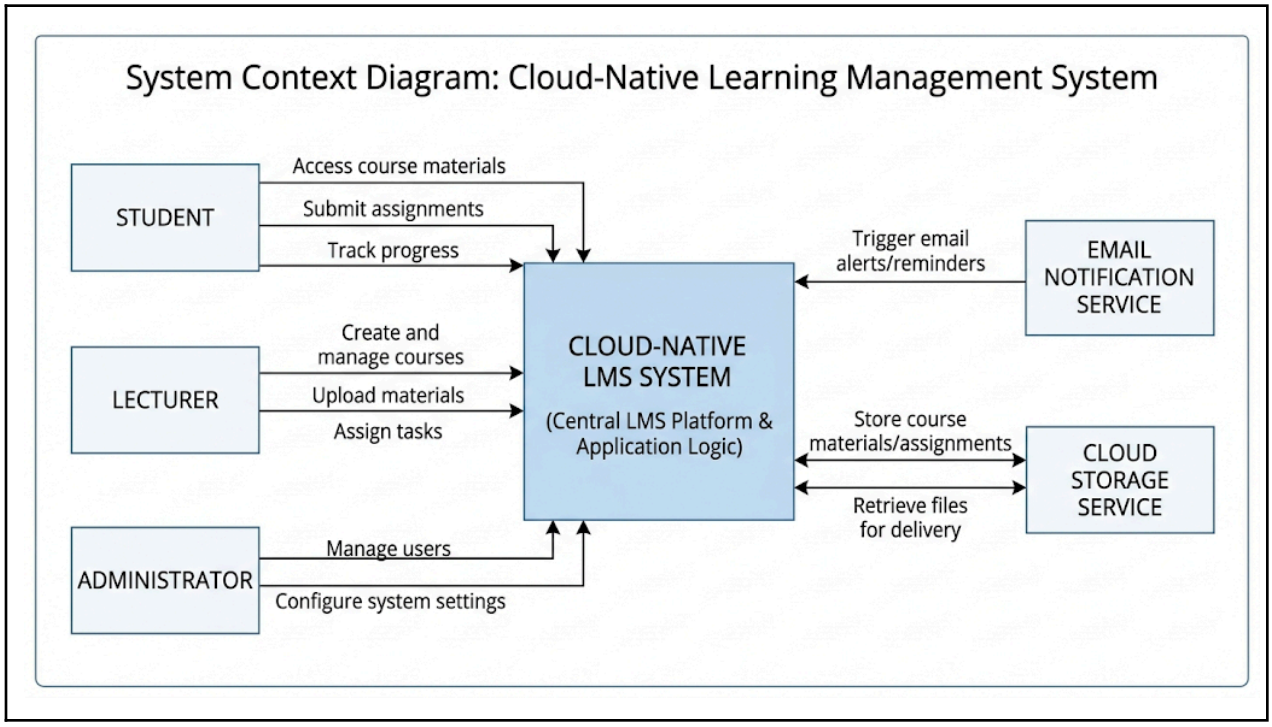

Figure 3 : System Context Diagram for Cloud-Native Learning Management System.

The System Context Diagram illustrates the high-level interaction between the Cloud-Native LMS and its external entities, including students, lecturers, and administrators. It emphasizes connection with other services like cloud storage and email notification systems. This diagram provides an overview of how users and external systems interact with the LMS without exposing internal system details.

11.2 COMPONENT / SERVICES ARCHITECTURE DIAGRAM

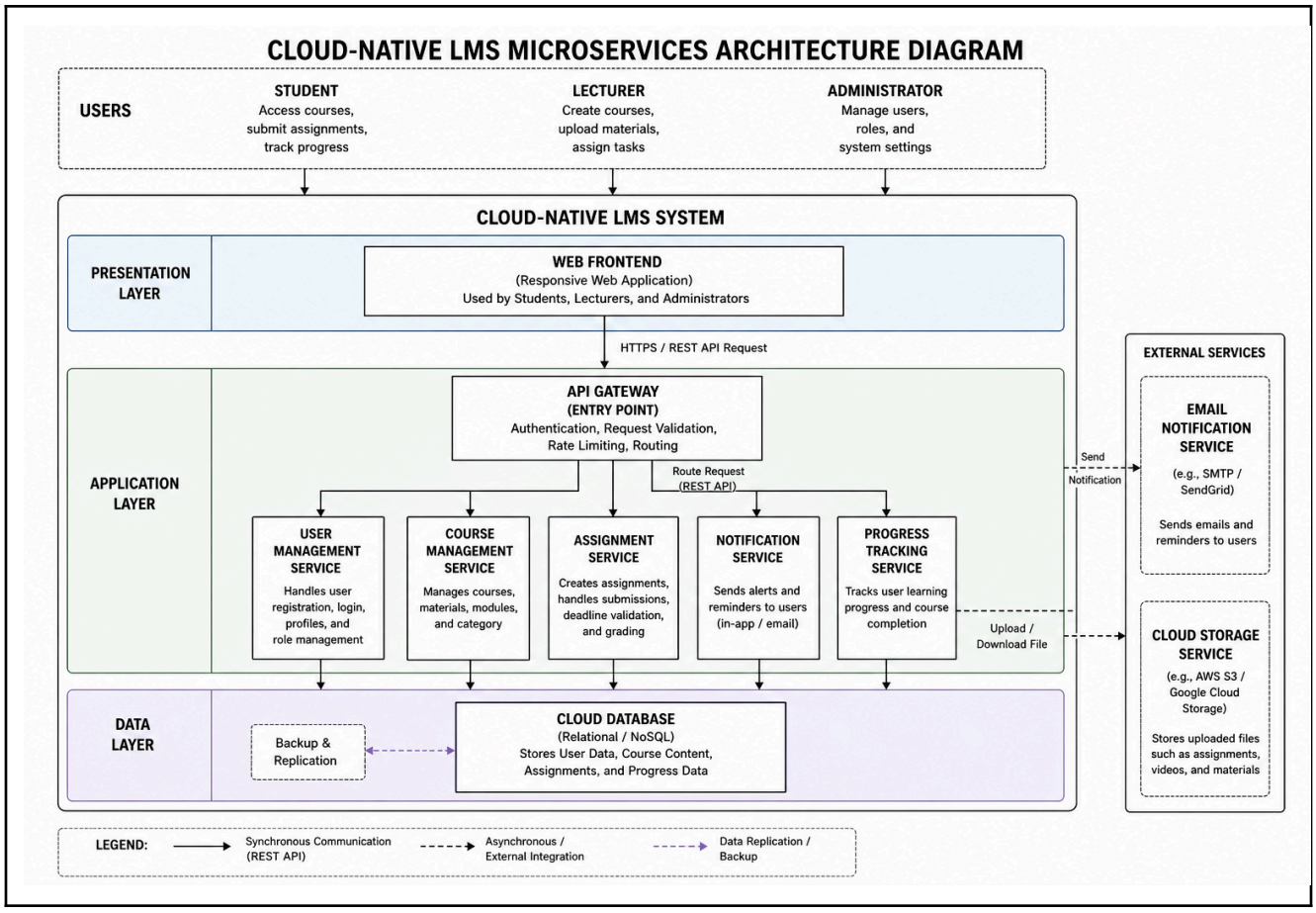

Figure 4 : Component Diagram for Cloud-Native LMS Microservices Architecture Diagram

Based on a layered design approach, the Cloud-Native LMS internal structure is depicted in the Component/Microservices design Diagram. The system is divided into Presentation, Application, and Data layers which each layer has their duties.

In the Presentation Layer, the Web Frontend serves as the user interface for students, lecturers, and administrators. This layer handles all user interactions including managing courses, submitting assignments and accessing course materials. RESTful API queries are used by the frontend to interface with the backend system over HTTPS.

The Application Layer acts as the core of the system, where all business logic is implemented. An API Gateway is used as the central entry point that handles authentication, request validation, and routing of incoming requests to the appropriate microservices. The User Management

Service, Course Management Service, Assignment Service, Notification Service, and Progress Tracking Service are among the independent microservices that make up the system. The flexibility and modularity of the system are increased since each microservice is in charge of a distinct function and can run, scale and be maintained independently.

In the Data Layer, a centralized Cloud Database is used to store all persistent data such as user information, course content, assignments, and progress tracking records. This layer facilitates high availability and guarantees effective data management.

In addition, the system integrates with external services to enhance functionality. While the Notification Service communicates with an Email Notification Service to send users notifications and reminders, the Assignment Service works with a Cloud Storage Service to manage file uploads and downloads.

Overall, all things considered the architecture shows a scalable, modular and maintainable system design that makes use of microservices and cloud-native concepts to meet contemporary e-learning needs.

11.3 INTERACTION DIAGRAM

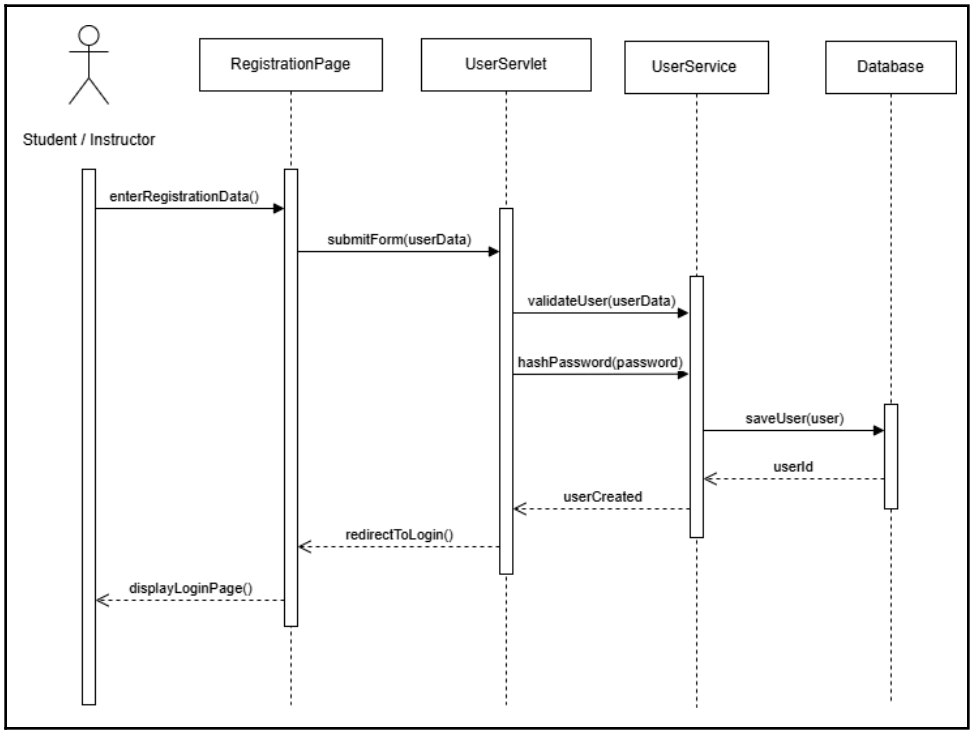

Figure 5 : User Management (User Registration)

The sequence diagram for the User Registration module shows how data flows from the front end to the database. It starts with the Student or Instructor entering details on the RegistrationPage which sends the data to the UserServlet. The servlet passes it to the UserService for validation and password hashing and then the service saves the new user in the Database. Once confirmed with a unique ID, the servlet signals success and redirects the user to the RegistrationPage to display the login screen, completing the workflow.

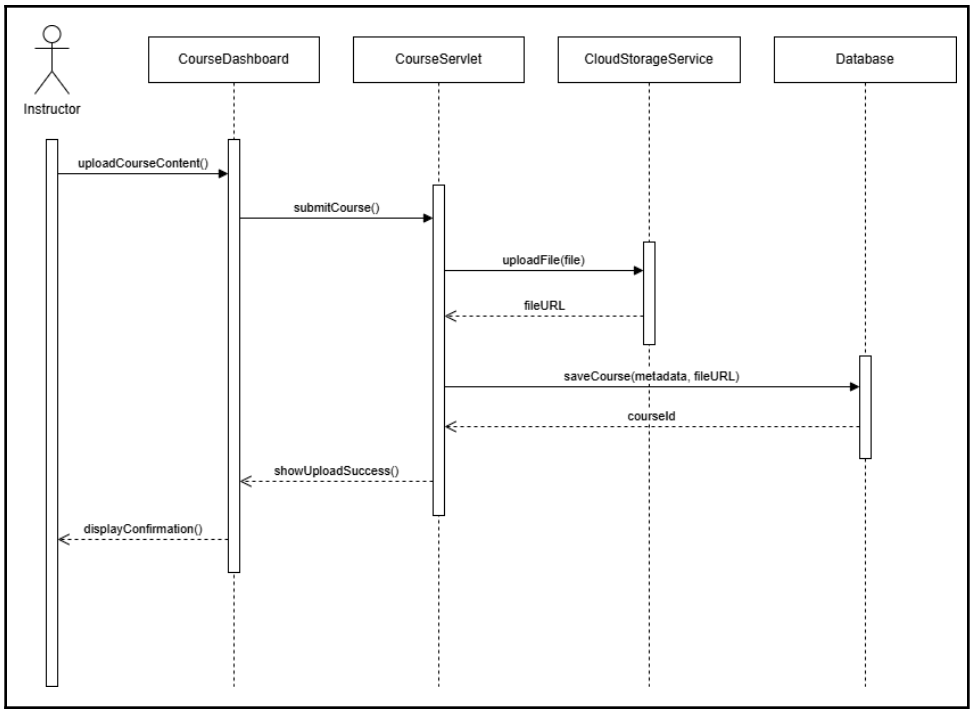

Figure 6 : Course Management (Create New Course)

The sequence diagram for the Course Management module shows how course content is uploaded. The process begins when the Instructor submits a file and metadata through the CourseDashboard, which forward the request to the CourseServlet. The servlet uploads the file to the CloudStorageService and receives a unique fileURL. It then stores the metadata and URL in the Database. Once the Database confirms with a courseId, the system notifies the Instructor by displaying a confirmation message on the dashboard.

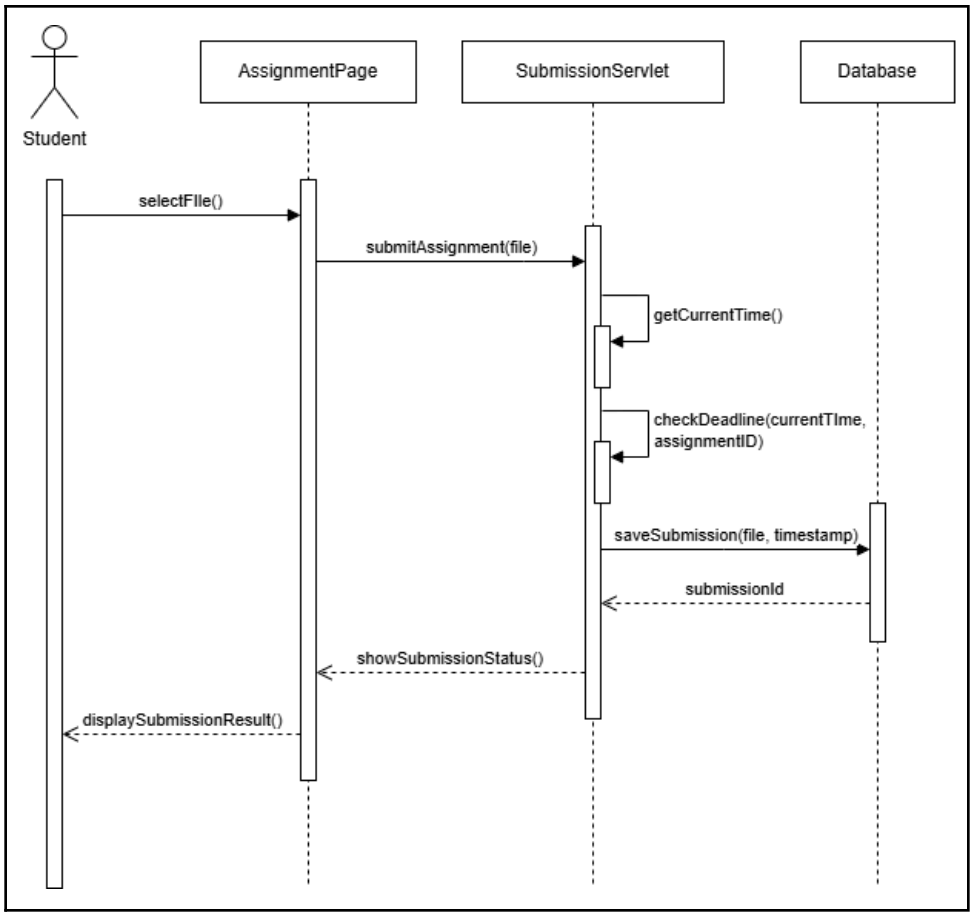

Figure 7 : Assignment & Submission (Student Upload)

The sequence diagram for the Assignment & Submission module shows how a student submits a file. The process starts when the Student selects a file on the AssignmentPage, which sends the request to the SubmissionServlet. The servlet checks the server time against the assignment deadline to ensure validity. If the submission is on time, it saves the file and timestamp in the Database. Once the Database confirms with a submissionId, the servlet updates the page to display the final submission result to the student.

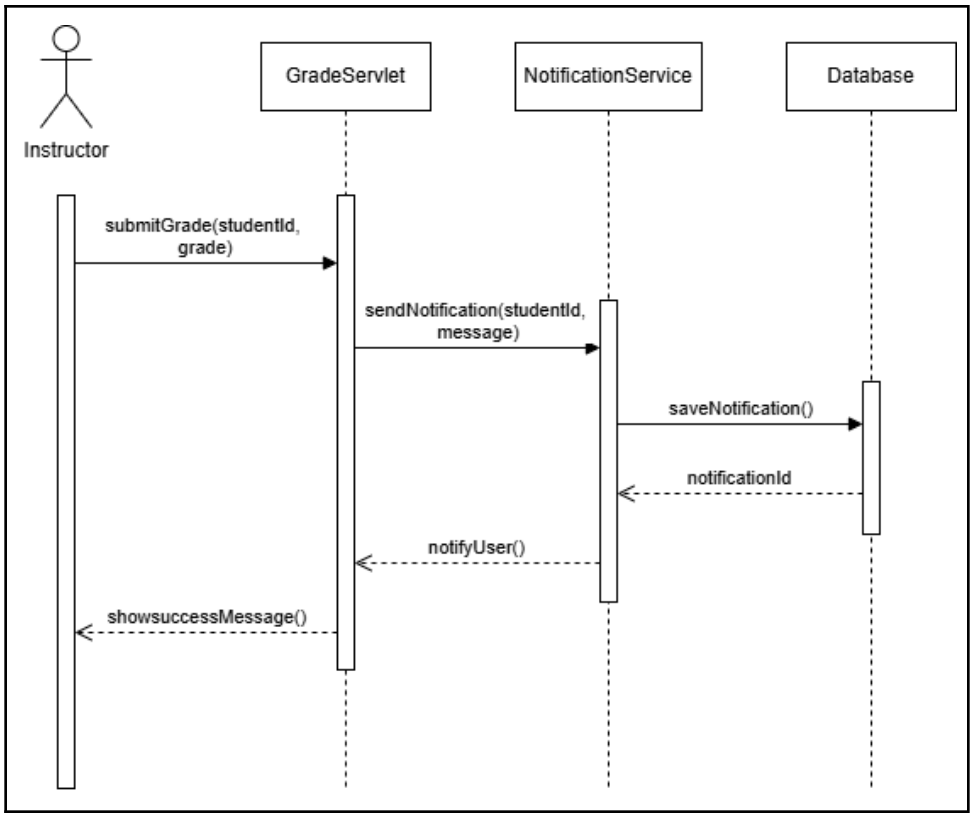

Figure 8 : Notification Module (Trigger Grade Alert)

The sequence diagram for the Notification Module shows how a student is alerted when an instructor posts a grade. It begins with the Instructor submitting a grade through the GradeServlet, which triggers the NotificationService to manage the alert. The service stores the notification details in the Database and once confirmed with a notificationId, it signals back to the servlet. Finally, the servlet displays a success message to the instructor.

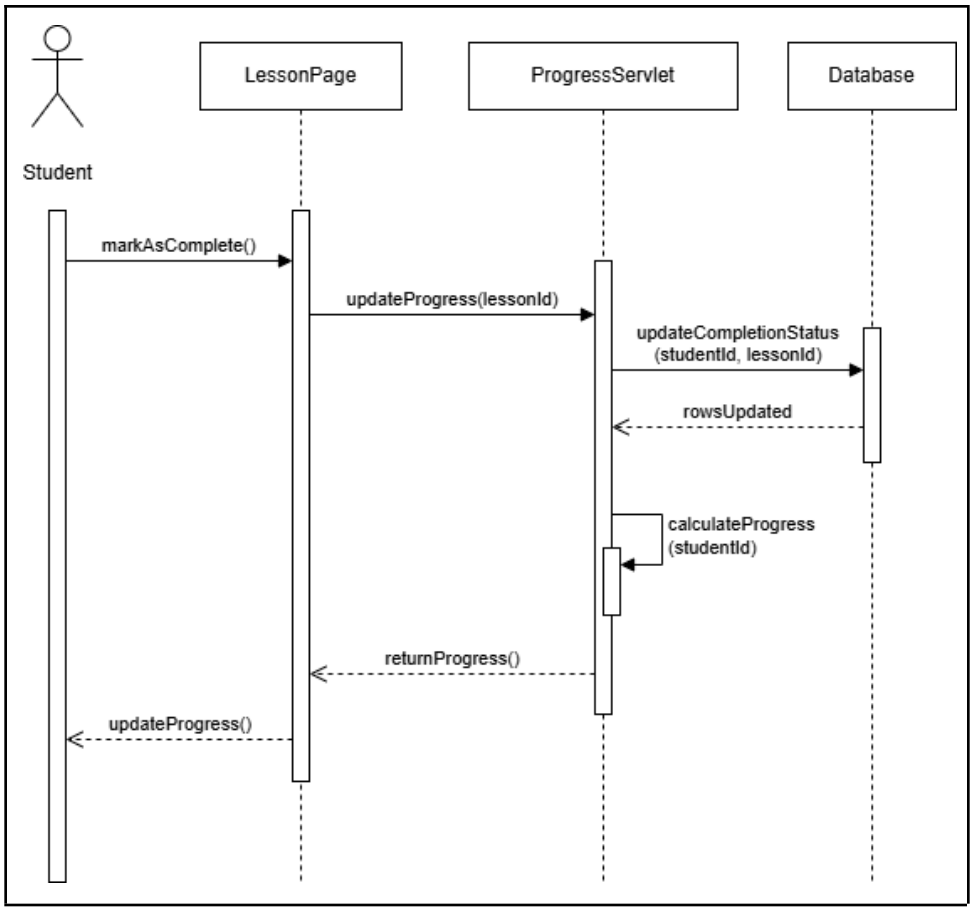

Figure 9 : Process Tracking (Update Completion)

The sequence diagram for the Progress Tracking module shows how a student's course completion status is updated. The process starts when the Student selects "markAsComplete()" on the LessonPage, which sends the request to the ProgressServlet. The servlet updates the completion status in the Database, and once confirmed, it calculates the student's new overall progress percentage. This updated value is then returned to the page, refreshing the progress bar for the student.

11.4 DEPLOYMENT DIAGRAM

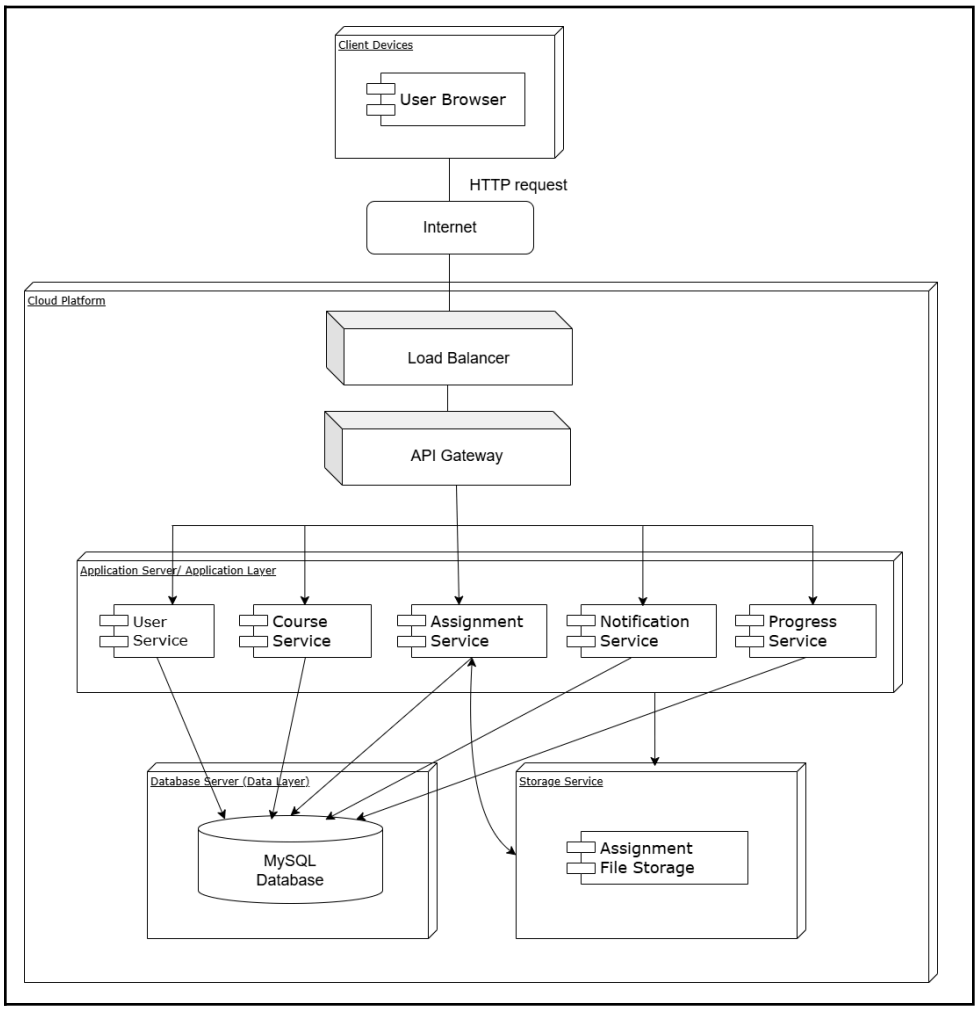

Figure 10 : Deployment DIagram of Cloud-Native Learning Management System (LMS)

The deployment diagram illustrates the distribution of the Cloud-Native LMS across a cloud infrastructure environment. At the client layer, users access the system using web browser on devices such as desktops, tablets, and smartphones. All communication between clients and the system are conducted with secure HTTP over the internet.

User requests are first handled by a Load Balancer which distributes traffic evenly across available backend resources to prevent

system overload during peak usage periods. The request then routed to an API Gateway which responsible for request routing, authentication and API management.

Within the application layer, the system is composed of multiple microservices, including User Management, Course Management, Assignment, Notification, and Progress Tracking services. Each microservice is deployed via containers using Docker technology, ensuring isolation and portability. The backend services interact with a centralized cloud database, which stores all persistent data such as user accounts, course materials, assignments, and progress records. The database is designed to support high availability and efficient data retrieval. Additionally, external services such as cloud storage for file uploads and email notification systems may be integrated to enhance system functionality.

12. REFERENCES

Colman, H., & Colman, H. (2026, March 20). LMS Requirements Checklist: Essential features & Capabilities (2026). *iSpring knowledge hub: practical insights to evaluate LMS solutions – Boost your L&D expertise. Get instructional design best practices, eLearning how to and case studies. Gain the best of your LMS and authoring tools*. https://www.ispring.com/knowledge-hub/lms-requirement

IBM. (n.d.). *Service component architecture (SCA)*. https://www.ibm.com/docs/en/cics-ts/6.x?topic=applications-service-component-architecture-sca

LMSpedia. (2026, February 21). *LMS architecture explained: Cloud, microservices & integration guide for enterprises*. https://lmspedia.org/lms-architecture-explained/

Microsoft. (n.d.). *What is cloud-native?* Microsoft Learn. Retrieved April 18, 2026, from https://learn.microsoft.com/en-us/dotnet/architecture/cloud-native/definition

Understanding cloud-native apps. (n.d.). https://www.redhat.com/en/topics/cloud-native-apps

Morgenthal, J. P. (2009, May 2). *What's the difference between a software component and A service?* https://jpmorgenthal.com/2009/05/02/whats-the-difference-between-a-software-component-and-a-service/

Pointer HR. (n.d.). *Software project team roles and responsibilities*. Retrieved April 11, 2026, from https://pointer.hr/software-project-team-roles-and-responsibilities/

ScienceDirect. (n.d.). *Layered architecture.* https://www.sciencedirect.com/topics/engineering/layered-architecture

Slideshare. (n.d.). *System overview diagram*. https://www.slideshare.net/slideshow/system-overview-diagram/88862926

Visual Paradigm. (n.d.). *What is a deployment diagram?* https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-deployment-diagram/

Visual Paradigm. (n.d.). *What is an interaction overview diagram?* https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-interaction-overview-diagram/

API Management - Amazon API Gateway - AWS. (n.d.). Amazon Web Services, Inc. https://aws.amazon.com/api-gateway/